

Avviso per tutti i docenti e i responsabili dei corsi

A seguito di un importante aggiornamento della piattaforma, attualmente in corso, vi chiediamo di **non duplicare i quiz contenenti domande random** fino a nuova comunicazione.

Questa misura è necessaria per consentirci di completare l'aggiornamento e risolvere le problematiche riscontrate nel minor tempo possibile.

Vi ringraziamo per la collaborazione.



Ingegneria del Software

[Torna al corso](#)

Stato	Completato
Iniziato	mercoledì, 1 luglio 2026, 11:58
Terminato	mercoledì, 1 luglio 2026, 12:13
Tempo impiegato	15 min. 10 secondi
Punteggio	49,00/50,00
Valutazione	98,00 su un massimo di 100,00

Domanda 1

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di consistenza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti. ✓
- ☐ 3. Assicurarsi che per ogni requisito esista un insieme di test che lo possa verificare.

Your answer is correct.

La risposta corretta è: Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Domanda 2

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di realismo" (realizability) che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che le funzionalità richieste al sistema siano necessarie per soddisfare le necessità del customer.
- ☐ 2. Assicurarsi che le performance richieste al sistema siano necessarie per soddisfare le necessità del customer.
- ☒ 3. Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfi i requisiti. ✓

Your answer is correct.

La risposta corretta è: Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfi i requisiti.

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente funzione di transizione.

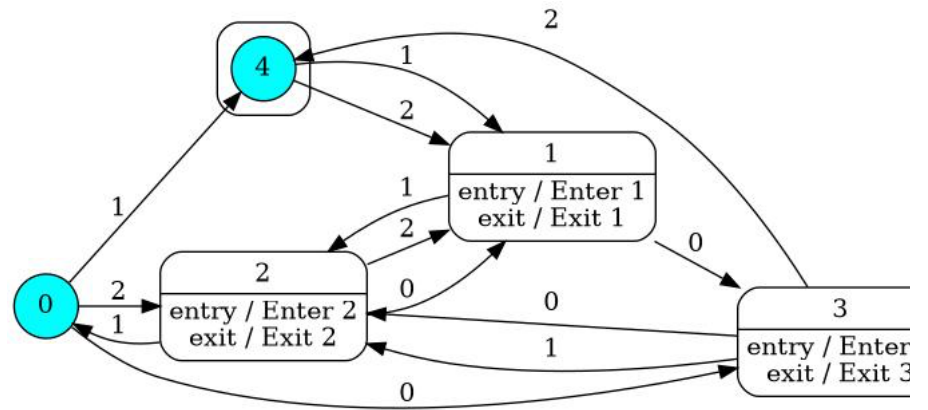
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (2);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (1);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    default: return (0);
  }
} // f()
```

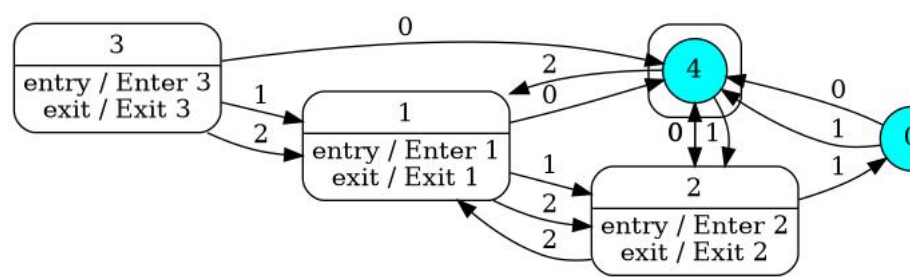
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

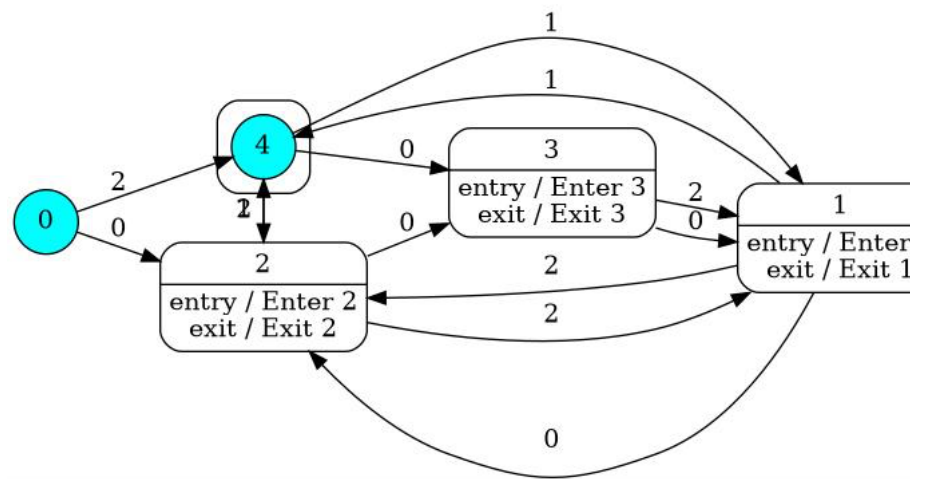
☐ 1.



☐ 2.

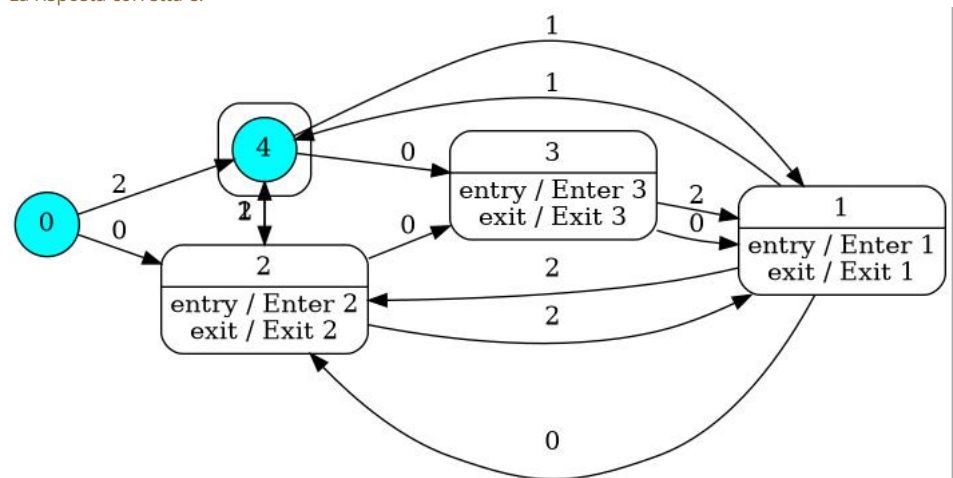


☒ 3.



Your answer is correct.

La risposta corretta è:



Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive il criterio di "requirements verifiability" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Per ciascuna coppia di componenti, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che l'interazione tra le componenti soddisfa tutti i requisiti di interfaccia.
- ☒ 2. Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato. 
- ☐ 3. Per ciascuna componente del sistema, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che essa soddisfa tutti i requisiti.

Your answer is correct.

La risposta corretta è: Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il **branch coverage** di un insieme di test cases è la percentuale di branch del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la branch coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((1)*x[0] + (3)*x[1] + (-4)*x[2] <= 1)
        if ((3)*x[0] + (3)*x[1] + (2)*x[2] <= -1)
            {return (0); }
        else
            {return (1); }
    else if ((0)*x[0] + (-3)*x[1] + (-2)*x[2] <= -4)
    {
        if ((-3)*x[0] + (-2)*x[1] + (1)*x[2] <= 0)
            {return (2); }
        else
            {return (3); }
    }
    else
        {return (4); }
} /* f() */
```

Test cases:

test 0: x[0] = -2, x[1] = 2, x[2] = -4,
test 1 : x[0] = -3, x[1] = -3, x[2] = 4,
test 2: x[0] = 2, x[1] = 4, x[2] = 0,
test 4: x[0] = 5, x[1] = 4, x[2] = 5,

Scegli un'alternativa:

- ☒ 1. Branch coverage: 80% ✓
- ☐ 2. Branch coverage: 50%
- ☐ 3. Branch coverage: 30%

Your answer is correct.

La risposta corretta è: Branch coverage: 80%

Domanda 6

Quale delle seguenti affermazione è vera riguardo al beta testing ?

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☐ 1. Test automatizzato sono eseguiti sulla versione finale del sistema presso il cliente.
- ☐ 2. Test automatizzato sono eseguiti sulla versione finale del sistema presso il sito di sviluppo del software.
- ☒ 3. Una release del software è resa disponibile agli utenti (beta users) per permettergli di sperimentare e quindi segnalare eventuali problemi rilevati agli sviluppatori. 

Your answer is correct.

La risposta corretta è: Una release del software è resa disponibile agli utenti (beta users) per permettergli di sperimentare e quindi segnalare eventuali problemi rilevati agli sviluppatori.

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((-2)*x[0] + (1)*x[1] + (0)*x[2] <= 1)
        if ((4)*x[0] + (0)*x[1] + (0)*x[2] <= -4)
            {return (0); }
        else
            {return (1); }
    else if ((-1)*x[0] + (1)*x[1] + (0)*x[2] <= 1)
        if ((-1)*x[0] + (3)*x[1] + (2)*x[2] <= 1)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = -2, x[1] = -2, x[2] = -1.

Test 2: x[0] = -1, x[1] = -2, x[2] = 2.

Test 3: x[0] = -4, x[1] = -2, x[2] = 0.

Test 4: x[0] = 4, x[1] = -1, x[2] = -1.

Scegli un'alternativa:

- ☐ 1. Coverage: 0.943
- ☐ 2. Coverage: 0.580
- ☒ 3. Coverage: 0.800  Correct.

Your answer is correct.

La risposta corretta è: Coverage: 0.800

Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il **branch coverage** di un insieme di test cases è la percentuale di branch del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la branch coverage con i tests cases dati.

```
-----  
int f(int x[3])  
{  
    if ((-3)*x[0] + (4)*x[1] + (-1)*x[2] <= 1)  
        if ((0)*x[0] + (2)*x[1] + (4)*x[2] <= 4)  
            {return (0); }  
        else  
            {return (1); }  
    else if ((1)*x[0] + (-2)*x[1] + (-3)*x[2] <= 3)  
    {  
        if ((2)*x[0] + (-3)*x[1] + (2)*x[2] <= 0)  
            {return (2); }  
        else  
            {return (3); }  
    }  
    else  
        {return (4); }  
} /* f() */
```

Test cases:

test 0, x[0] = 0, x[1] = 1, x[2] = 1,

test 1, x[0] = -2, x[1] = 4, x[2] = -1,

test 2, x[0] = 1, x[1] = 2, x[2] = -4,

test 3, x[0] = 5, x[1] = 4, x[2] = 1.

Scegli un'alternativa:

- ☒ 1. 50% 
- ☐ 2. 90%
- ☐ 3. 80%

Your answer is correct.

La risposta corretta è: 50%

Domanda 9

Quale dei seguenti è un requisito non funzionale?

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☒ 1. Il tempo di completamento della funzione è al più 2ms. ✔ Correct.
- ☐ 2. L'output della funzione è sempre positivo.
- ☐ 3. L'output della funzione è sempre un numero paro.

Your answer is correct.

La risposta corretta è: Il tempo di completamento della funzione è al più 2ms.

Domanda 10

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo 10000 Eur e deve essere ripetuta una seconda volta con probabilità 0.1. Qual'è il costo atteso dello sviluppo dell'intero software?

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☒ 1. 22000 ✔
- ☐ 2. 25000
- ☐ 3. 30000

Your answer is correct.

La risposta corretta è: 22000

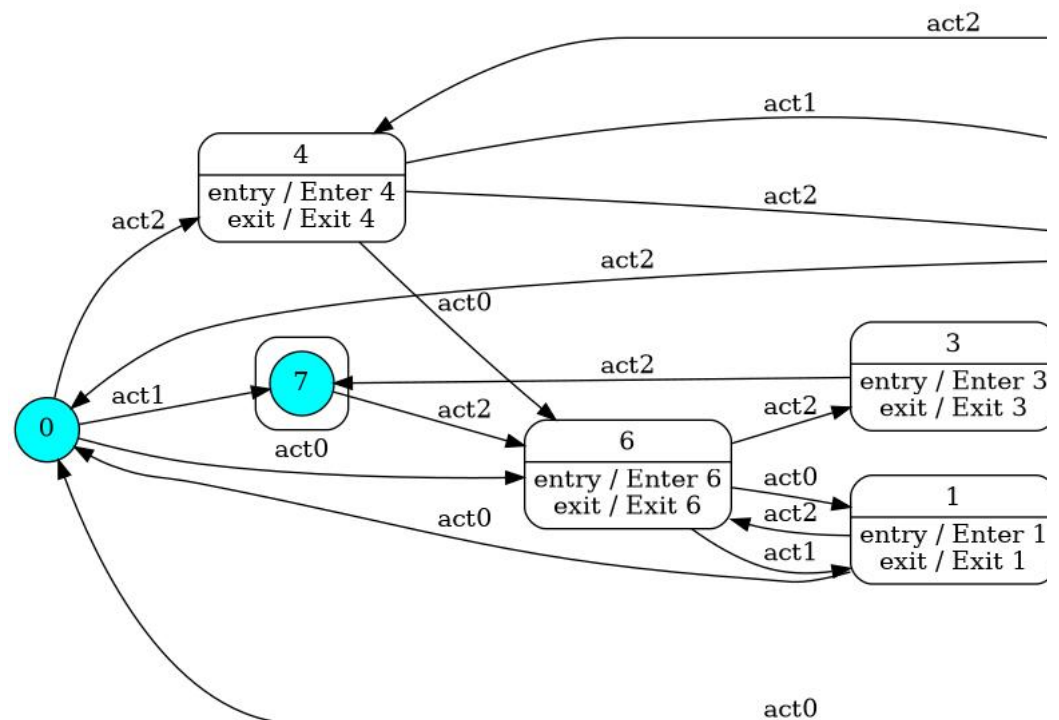
Domanda 11

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act2 act0 act2 act0 act0

Test case 2: act1

Test case 3: act0 act0 act2 act1

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. State coverage: 0.875 Correct.
- ☐ 2. State coverage: 0.213
- ☐ 3. State coverage: 0.475

Your answer is correct.

La risposta corretta è: State coverage: 0.875

Domanda 12

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale dei seguenti è un requisito non funzionale?

Scegli un'alternativa:

- ☒ 1. L'esecuzione della funzione occupa al più 2MB di RAM. ✔ Correct.
- ☐ 2. L'output della funzione è un vettore ordinato in modo ascendente.
- ☐ 3. L'output della funzione è un vettore della stessa dimensione del vettore di input.

Your answer is correct.

La risposta corretta è: L'esecuzione della funzione occupa al più 2MB di RAM.

Domanda 13

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti affermazioni è vera riguardo all'alpha testing ?

Scegli un'alternativa:

- ☒ 1. Gli utenti del sistema lavorano insieme al team di sviluppo per testare il software nel sito di sviluppo. ✔
- ☐ 2. Test automatizzati sono eseguiti sulla prima release del sistema.
- ☐ 3. Test automatizzati sono eseguiti su una versione preliminare del sistema.

Your answer is correct.

La risposta corretta è: Gli utenti del sistema lavorano insieme al team di sviluppo per testare il software nel sito di sviluppo.

Domanda 14

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il system testing si concentra su:

Scegli un'alternativa:

- ☐ 1. Testare le funzionalità di unità software individuali, oggetti, classi o metodi.
- ☐ 2. Testare le interfacce per ciascuna componente.
- ☒ 3. Testare l'interazione tra le componenti del sistema (cioè, integrazione di molte unità di sistema). ✔

Your answer is correct.

La risposta corretta è: Testare l'interazione tra le componenti del sistema (cioè, integrazione di molte unità di sistema).

Domanda 15

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante note da dati storici aziendali.

- **$b = 0.000021$**
- **$C = 710301.000000$**
- **$R = 493.000000$**

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☒ 1. Costo: 342379  Correct.
- ☐ 2. Costo: 125277
- ☐ 3. Costo: 392831

Your answer is correct.

La risposta corretta è: Costo: 342379

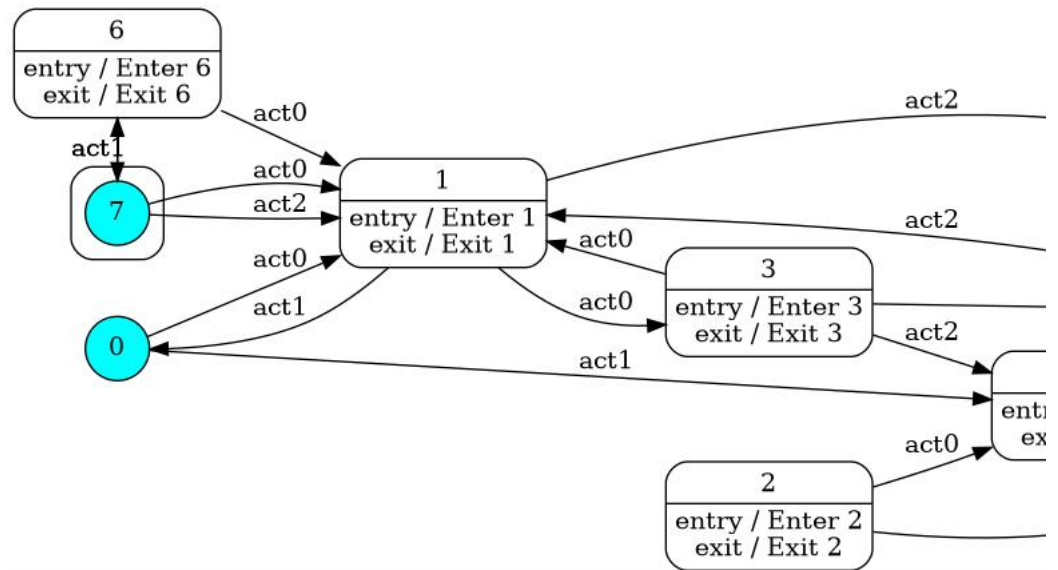
Domanda 16

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act1 act0 act2 act0 act2

Test case 2: act0 act0 act1 act2 act2

Test case 3: act0 act1 act0 act0 act1

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.331
- ☒ 2. State coverage: 0.625 Correct.
- ☐ 3. State coverage: 0.127

Your answer is correct.

La risposta corretta è: State coverage: 0.625

Domanda 17

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

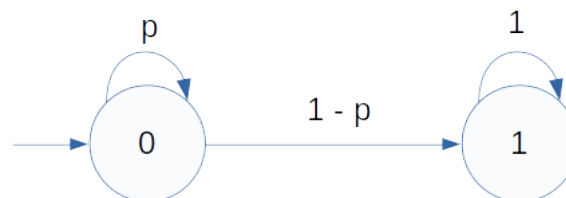
Si consideri il processo software con due fasi (0 ed 1) rappresentato con la Markov chain in figura. Lo stato iniziale 0 e p è in $(0, 1)$. Il costo dello stato (fase) x è $c(x)$. La fase 0 è la fase di design, che ha probabilità p di dover essere ripetuta causa errori. La fase 1 rappresenta il completamento del processo software, e quindi $c(1) = 0$.

Il costo di una istanza del processo software descritto sopra è la somma dei costi degli stati attraversati (tenendo presente che si parte sempre dallo stato 0).

Quindi il costo $C(X)$ della sequenza di stati $X = x(0), x(1), x(2), \dots$ è $C(X) = c(x(0)) + c(x(1)) + c(x(2)) + \dots$

Ad esempio se $X = 0, 1$ abbiamo $C(X) = c(0) + c(1) = c(0)$ (poiché $c(1) = 0$).

Quale delle seguenti formule calcola il valore atteso del costo per completare il processo software di cui sopra



Scegli un'alternativa:

- ☐ 1. $c(0) \cdot (1 - p) / p$
- ☐ 2. $c(0) / (p \cdot (1 - p))$
- ☒ 3. $c(0) / (1 - p)$ ✓

Your answer is correct.

La risposta corretta è: $c(0) / (1 - p)$

Domanda 18

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il seguente frammento di pseudo codice.

```

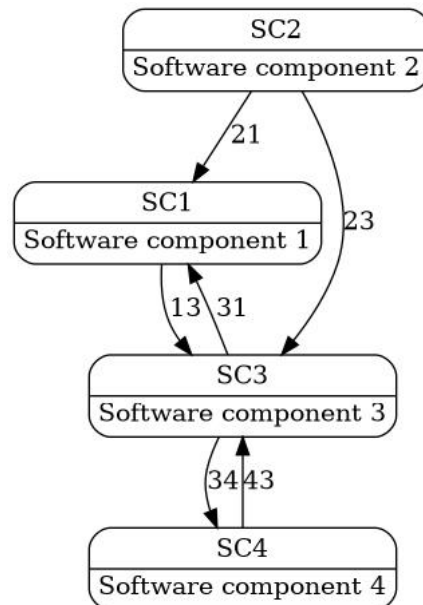
// Begin input/output connections for system components
Input 13 of SC3 = Output 13 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
  
```

Quale delle seguenti architetture software meglio lo rappresenta ?

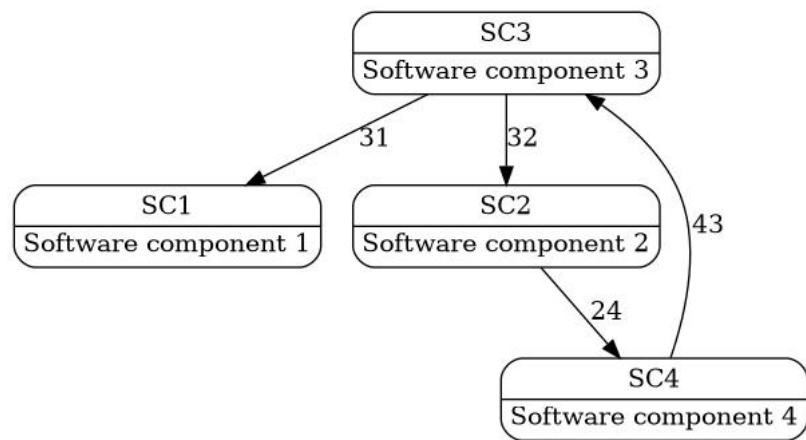
Scegli un'alternativa:

1.

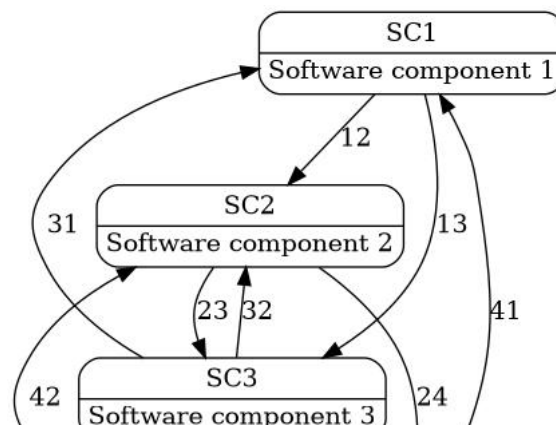
Correct.



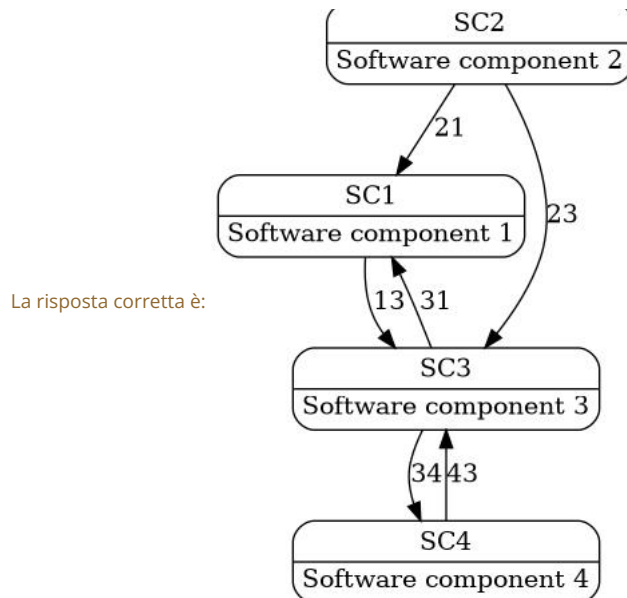
2.



3.



Your answer is correct.



Domanda 19

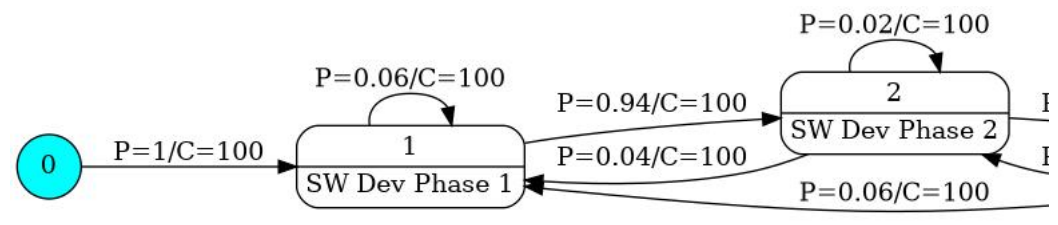
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.1066 ✔ Correct.
- ☐ 2. 0.5533
- ☐ 3. 0.07462

Your answer is correct.

La risposta corretta è: 0.1066

Domanda 20

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

"Ogni giorno, per ciascuna clinica, il sistema genererà una lista dei pazienti che hanno un appuntamento quel giorno."

La frase precedente è un esempio di:

Scegli un'alternativa:

- ☒ 1. Requisito funzionale. ✓
- ☐ 2. Requisito di performance.
- ☐ 3. Requisito non-funzionale.

Your answer is correct.

La risposta corretta è: Requisito funzionale.

Domanda 21

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((2)*x[0] + (-3)*x[1] + (-1)*x[2] <= -2)
        if ((-2)*x[0] + (1)*x[1] + (-1)*x[2] <= 3)
            {return (0); }
        else
            {return (1); }
    else if ((1)*x[0] + (3)*x[1] + (2)*x[2] <= 4)
        if ((3)*x[0] + (1)*x[1] + (2)*x[2] <= -4)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 2, x[1] = 2, x[2] = 1.

Test 2: x[0] = -1, x[1] = -3, x[2] = -1.

Test 3: x[0] = 5, x[1] = 2, x[2] = 0.

Test 4: x[0] = 3, x[1] = -3, x[2] = 1.

Scegli un'alternativa:

- ☒ 1. Coverage: 0.800 ✓ Correct.
- ☐ 2. Coverage: 0.253
- ☐ 3. Coverage: 0.413

Your answer is correct.

La risposta corretta è: Coverage: 0.800

Domanda 22

Quale delle seguenti affermazioni è vera riguardo al performance testing?

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☒ 1. Il performance testing è tipicamente eseguito una volta che il sistema è stato completamente integrato. 
- ☐ 2. Il performance testing è tipicamente eseguito su un prototipo del sistema.
- ☐ 3. Il performance testing è tipicamente eseguito solo sulle componenti del sistema prima dell'integrazione.

Your answer is correct.

La risposta corretta è: Il performance testing è tipicamente eseguito una volta che il sistema è stato completamente integrato.

Domanda 23

Il component testing si concentra su:

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☒ 1. Testare le interfacce per ciascun componente. 
- ☐ 2. Testare l'interazione tra molte componenti (cioè integrazione di molte unità).
- ☐ 3. Testare funzionalità di unità software individuali, oggetti, classi o metodi.

Your answer is correct.

La risposta corretta è: Testare le interfacce per ciascun componente.

Domanda 24

La WBS (Work Breakdown Structure) di un progetto consiste di 5 tasks: T0, T1, T2, T3, T4.

Risposta non data

Punteggio max.: 1,00

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
4	0
4	1
4	2
4	3

La riga (*i*, *j*) nella tabella delle precedenze indica che il task *j* precede il task *i* (cioè che il task *i* può iniziare solo dopo che il task *j* è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
-	-

0	6
1	6
2	2
3	2
4	6

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 1 | 2 |
| 0 | 3 | 2 |
| 1 | 1 | 4 |
| 2 | 2 | 2 |
| 3 | 4 | 2 |
| 4 | 4 | 4 |
| 5 | 0 | 4 |
| 6 | 0 | 2 |

☐ 2.

Time slot (Month)	Task Effort (PM)	
0	1	4
1	1	2
1	2	1
2	0	2
2	3	2
3	0	4
4	4	4
5	4	2

☐ 3.

Time slot (Month)	Task Effort (PM)	
0	2	2
0	3	2
1	0	2
1	1	2
2	1	4
3	0	4
4	4	2
5	4	4

Your answer is incorrect.

La risposta corretta è:

Time slot (Month)	Task Effort (PM)	
0	2	2
0	3	2
1	0	2
1	1	2
2	1	4
3	0	4
4	4	2
5	4	4

Domanda 25

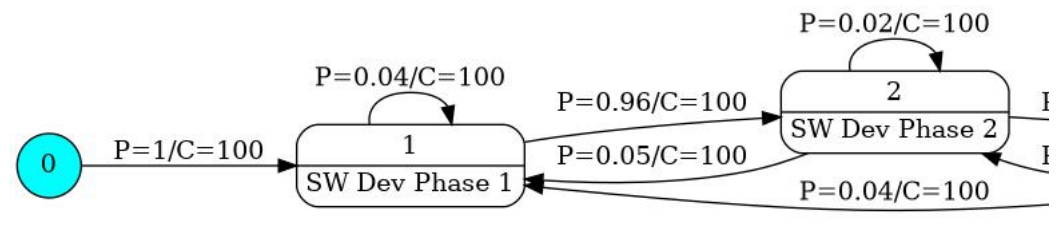
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.05068
- ☐ 2. 0.5362
- ☒ 3. 0.0724 ✓ Correct.

Your answer is correct.

La risposta corretta è: 0.0724

Domanda 26

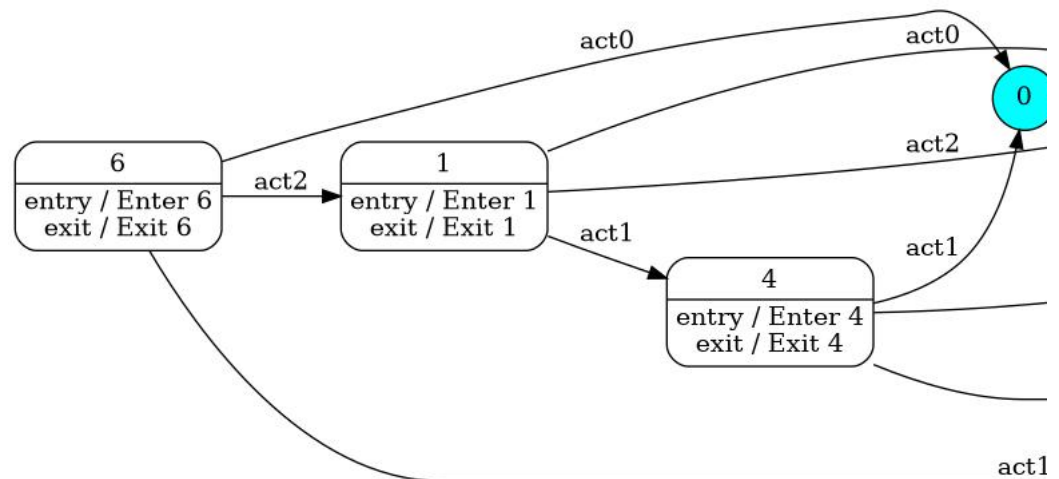
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act2 act1

Test case 2: act1

Test case 3: act2 act0 act0 act0

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. Transition coverage: 0.217 ✓ Correct.
- ☐ 2. Transition coverage: 0.053
- ☐ 3. Transition coverage: 0.110

Your answer is correct.

La risposta corretta è: Transition coverage: 0.217

Domanda 27

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quali delle seguenti attività è parte del processo di validazione dei requisiti ?

Scegli un'alternativa:

- ☒ 1. Accertarsi che i requisiti definiscano un sistema che risolve il problema che l'utente pianifica di risolvere. ✓
- ☐ 2. Accertarsi che l'architettura del sistema soddisfi i requisiti dati.
- ☐ 3. Accertarsi che il sistema soddisfi i requisiti dati.

Your answer is correct.

La risposta corretta è: Accertarsi che i requisiti definiscano un sistema che risolve il problema che l'utente pianifica di risolvere.

Domanda 28

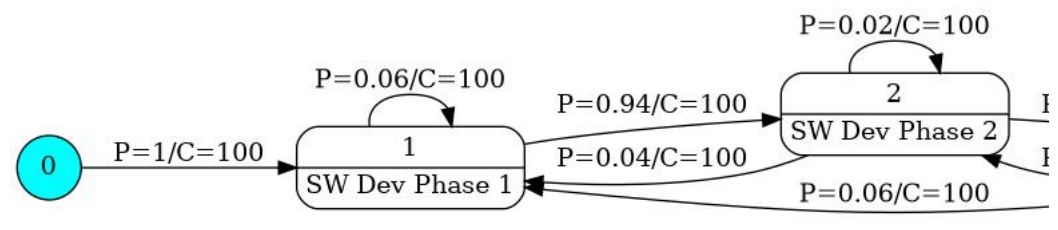
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.501001
- ☐ 2. 0.857858
- ☒ 3. 0.715716 ✓ Correct.

Your answer is correct.

La risposta corretta è: 0.715716

Domanda 29

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "validity check" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.
- ☒ 2. Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer". ✓
- ☐ 3. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.

Your answer is correct.

La risposta corretta è: Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer".

Domanda 30

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo agile consiste di varie iterazioni. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione e sia A il costo di una iterazione. Il valore atteso del costo per l'iterazione è:

Scegli un'alternativa:

- ☒ 1. $(1 + p) * A$ ✓
- ☐ 2. A
- ☐ 3. $p * A$

Your answer is correct.

La risposta corretta è: $(1 + p) * A$

Domanda 31

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

**Si consideri il Test-Driven Development (TDD).
Quale delle seguenti affermazioni è vera?**

Scegli un'alternativa:

- ☐ 1. Scrivi test automatizzati per tutti i requisiti di sistema, esegui i test e rivedi l'implementazione come necessario.
- ☒ 2. Per ciascun incremento di funzionalità, scrivi test automatizzati, implementa la funzionalità, esegui i test e rivedi l'implementazione come necessario. ✓
- ☐ 3. Per ciascun incremento di funzionalità, implementa la funzionalità, scrivi test automatizzati, esegui i test e rivedi l'implementazione come necessario.

Your answer is correct.

La risposta corretta è: Per ciascun incremento di funzionalità, scrivi test automatizzati, implementa la funzionalità, esegui i test e rivedi l'implementazione come necessario.

Domanda 32

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Pre-condizioni, invarianti e post-condizioni di un programma possono essere definiti usando la macro del C `assert()` (in `<assert.h>`). In particolare, `assert(expre)` non fa nulla se l'espressione `expre` vale **TRUE (cioè non è 0), stampa un messaggio di errore su `stderr` e abortisce l'esecuzione del programma altrimenti.**

Si consideri la funzione C

`int f(int x, int y) { ..... }`

Quale delle seguenti assert esprime l'invariante che le variabili locali `z` e `w` di `f()` hanno somma minore di 1 oppure maggiore di 7 ?

Scegli un'alternativa:

- ☐ 1. `int f(in x, int y)`
`{`
`int z, w;`
`assert( (z + w <= 1) || (z + w >= 7));`
`.....`
`}`
- ☒ 2.  `int f(in x, int y)`
`{`
`int z, w;`
`assert( (z + w < 1) || (z + w > 7));`
`.....`
`}`
- ☐ 3. `int f(in x, int y)`
`{`
`int z, w;`
`assert( (z + w > 1) || (z + w < 7));`
`.....`
`}`

Your answer is correct.

La risposta corretta è:

```
int f(in x, int y)

{
int z, w;
assert( (z + w < 1) || (z + w > 7));
.....
}
```

Domanda 33

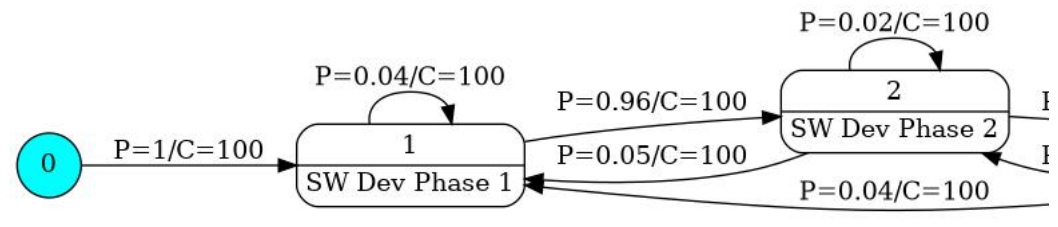
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.05607
- ☒ 2. 0.0801 ✔ Correct.
- ☐ 3. 0.54005

Your answer is correct.

La risposta corretta è: 0.0801

Domanda 34

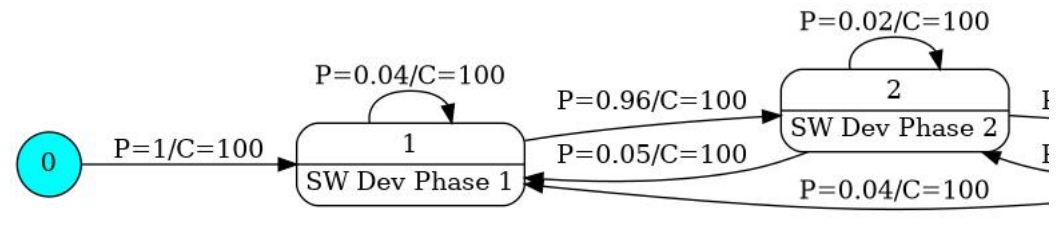
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.549965
- ☒ 2. 0.785664 ✓ Correct.
- ☐ 3. 0.892832

Your answer is correct.

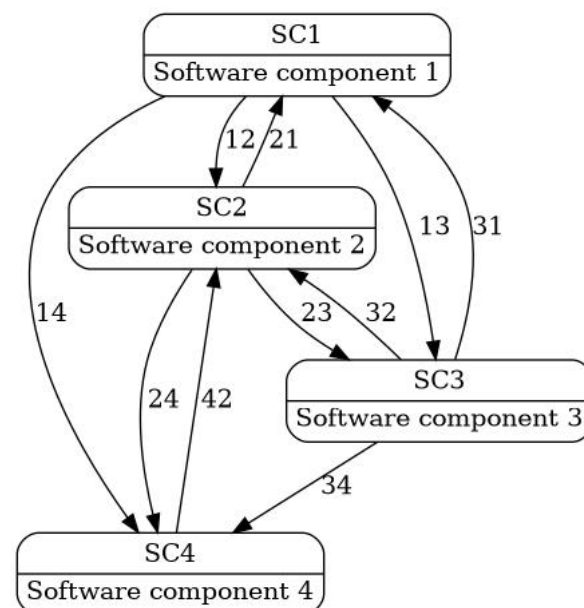
La risposta corretta è: 0.785664

Domanda 35

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☒ 1. // Begin input/output connections for system components ✔ Correct.
- Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 42 of SC2 = Output 42 of SC4
// End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
- Input 12 of SC2 = Output 12 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
- ☐ 3. // Begin input/output connections for system components
- Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3

```
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 42 of SC2 = Output 42 of SC4
// End input/output connections for system components
```

Domanda 36

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di completezza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer. ✓
- ☐ 3. Assicurarsi che per ogni requisito sia stato implementato nel sistema.

Your answer is correct.

La risposta corretta è: Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer.

Domanda 37

Risposta corretta

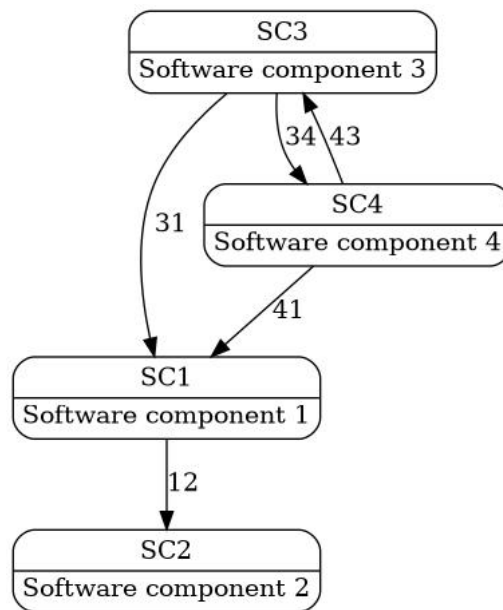
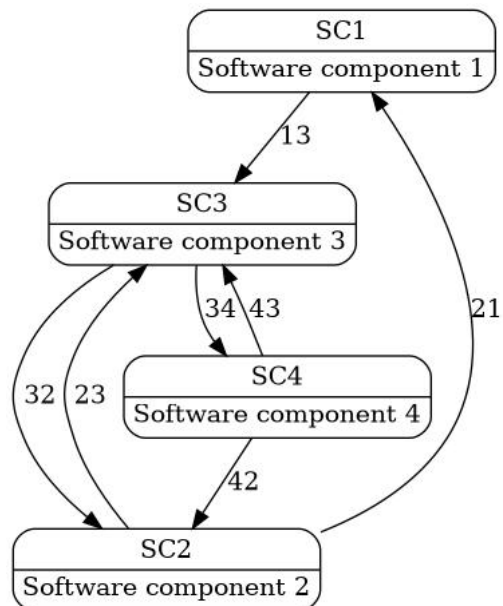
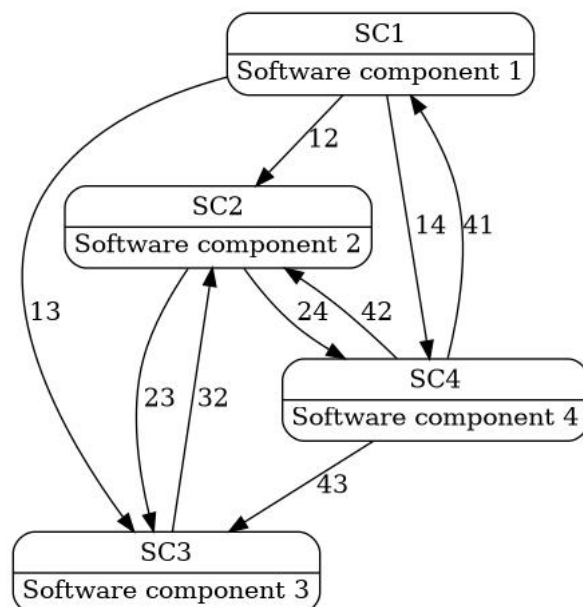
Punteggio ottenuto 1,00 su 1,00

Si consideri il seguente frammento di pseudo codice.

```
// Begin input/output connections for system components
Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 32 of SC2 = Output 32 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
```

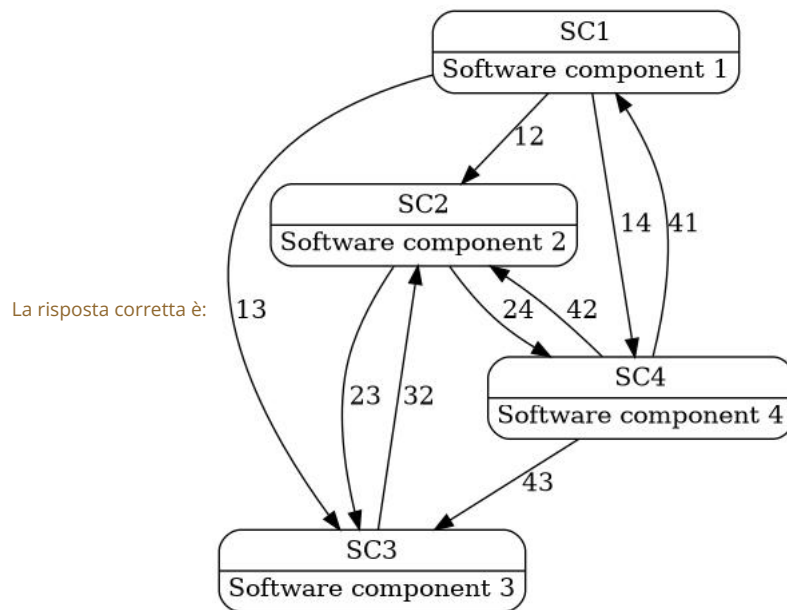
Quale delle seguenti architetture software meglio lo rappresenta ?

Scegli un'alternativa:

☐ 1.☐ 2.☒ 3.

✓ Correct.

Your answer is correct.



Domanda 38

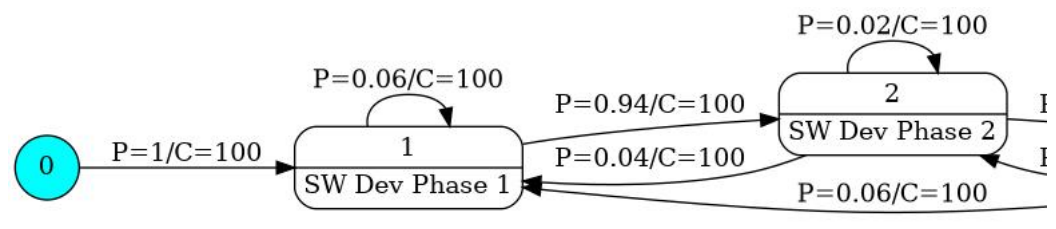
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.5398
- ☒ 2. 0.0796 ✔ Correct.
- ☐ 3. 0.05572

Your answer is correct.

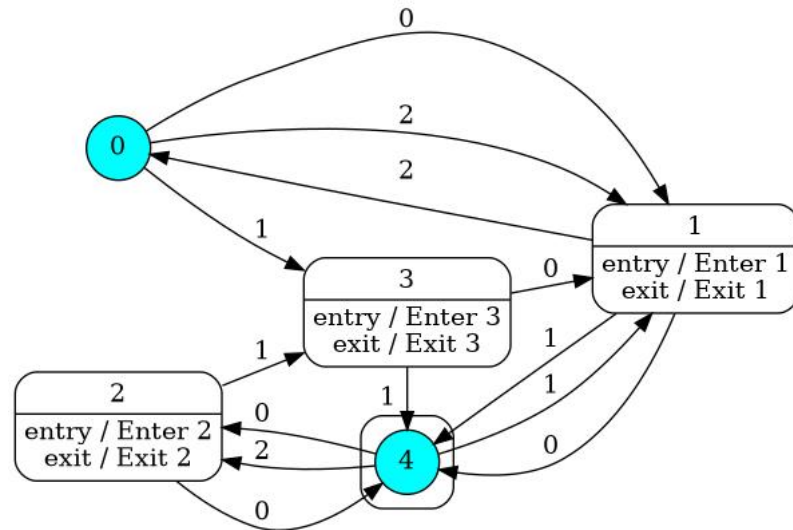
La risposta corretta è: 0.0796

Domanda 39

Si consideri lo state diagram in figura.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

☐ 1. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (4);
    case 1: return (3);
    case 2: return (1);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (0);
    case 1: return (2);
    case 2: return (1);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (1);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (1);
    case 1: return (2);
    case 2: return (1);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (2);
    case 1: return (4);
    case 2: return (1);
    default: return (0); }
  default: return (0); }
} // f()
```

☒ 2. `#include "main.h"`  Correct.

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (1);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    default: return (0);
}
} // f()
```

☐ 3. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    default: return (0);
}
} // f()
```

Your answer is correct.

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (1);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    default: return (0);
}
} // f()
```

La risposta corretta è:

Domanda 40

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante note da dati storici aziendali.

- **$b = 0.000015$**
- **$C = 591805.000000$**
- **$R = 649.000000$**

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☐ 1. Costo: 491756
- ☒ 2. Costo: 440932  Correct.
- ☐ 3. Costo: 668159

Your answer is correct.

La risposta corretta è: Costo: 440932

Domanda 41

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il **branch coverage** di un insieme di test cases è la percentuale di branch del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la branch coverage con i tests cases dati.

```
-----  
int f(int x[3])  
{  
    if ((2)*x[0] + (3)*x[1] + (2)*x[2] <= 3)  
        if ((1)*x[0] + (4)*x[1] + (4)*x[2] <= 0)  
            {return (0); }  
        else  
            {return (1); }  
    else if ((-2)*x[0] + (2)*x[1] + (3)*x[2] <= 3)  
    {  
        if ((-4)*x[0] + (-4)*x[1] + (-1)*x[2] <= 0)  
            {return (2); }  
        else  
            {return (3); }  
    }  
    else  
        {return (4); }  
} /* f() */  
-----
```

Test cases:

test 0: x[0] = 1, x[1] = 3, x[2] = 3,
test 1: x[0] = 5, x[1] = 4, x[2] = -2,
test 4: x[0] = -3, x[1] = 1, x[2] = -4,

Scegli un'alternativa:

- ☐ 1. Branch coverage: 30%
- ☐ 2. Branch coverage: 80%
- ☒ 3. Branch coverage: 60% ☺

Your answer is correct.

La risposta corretta è: Branch coverage: 60%

Domanda 42

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso. Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante note da dati storici aziendali. Quale sarà il costo dello sviluppo S di un software il cui costo della failure è C ed il rischio ammesso è R ?

Scegli un'alternativa:

- ☒ 1. $S = (1/b) \cdot \ln(C/R)$ ✓
- ☐ 2. $S = b \cdot \ln(R/C)$
- ☐ 3. $S = (1/b) \cdot \ln(R/C)$

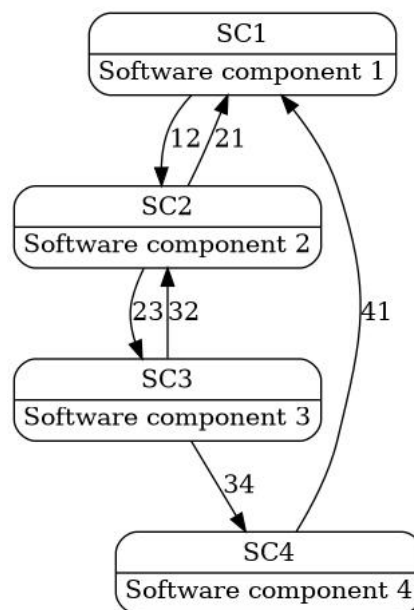
Your answer is correct.

La risposta corretta è: $S = (1/b) \cdot \ln(C/R)$ **Domanda 43**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☒ 1. // Begin input/output connections for system components  Correct.
- Input 12 of SC2 = Output 12 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
// End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
- Input 13 of SC3 = Output 13 of SC1
Input 41 of SC1 = Output 41 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
- ☐ 3. // Begin input/output connections for system components
- Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
// End input/output connections for system components

Domanda 44

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti affermazioni è vera riguardo ai metodi agile ?

Scegli un'alternativa:

- ☐ 1. I metodi agile sono metodi di sviluppo plan-driven.
- ☐ 2. I metodi agile sono metodi di sviluppo orientato al riuso.
- ☒ 3. I metodi agile sono metodi di sviluppo incrementale. 

Your answer is correct.

La risposta corretta è: I metodi agile sono metodi di sviluppo incrementale.

Domanda 45

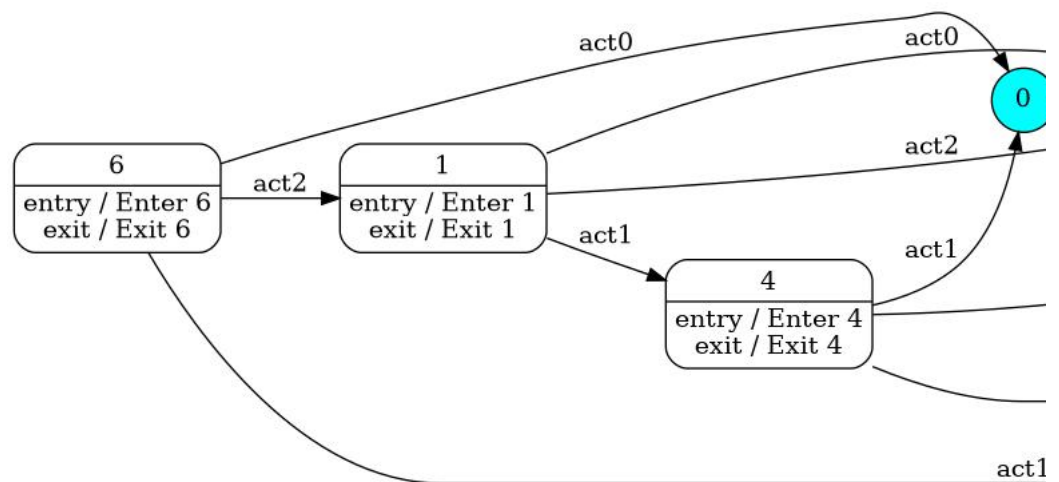
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act2 act1

Test case 2: act1

Test case 3: act2 act0 act0 act0

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. Transition coverage: 0.217 Correct.
- ☐ 2. Transition coverage: 0.053
- ☐ 3. Transition coverage: 0.110

Your answer is correct.

La risposta corretta è: Transition coverage: 0.217

Domanda 46

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Pre-condizioni, invarianti e post-condizioni di un programma possono essere definiti usando la macro del C `assert()` (in `<assert.h>`). In particolare, `assert(expre)` non fa nulla se l'espressione `expre` vale **TRUE (cioè non è 0), stampa un messaggio di errore su `stderr` e abortisce l'esecuzione del programma altrimenti.**

Si consideri la funzione C

`int f(int x, int y) { ..... }`

Quale delle seguenti assert esprime la pre-condizione che entrambi gli argomenti di `f` sono non-negativi ed almeno uno di loro è maggiore di 3 ?

Scegli un'alternativa:

- ☐ 1. `int f(in x, int y)`
`{`
`assert( (x > 0) && (y > 0) && ((x >= 3) || (y > 3)) );`
`.....`
`}`
- ☐ 2. `int f(in x, int y)`
`{`
`assert( (x >= 0) && (y >= 0) && ((x >= 3) || (y >= 3)) );`
`.....`
`}`
- ☒ 3. ✔
- `int f(in x, int y)`
`{`
`assert( (x >= 0) && (y >= 0) && ((x > 3) || (y > 3)) );`
`.....`
`}`

Your answer is correct.

La risposta corretta è:

```
int f(in x, int y)
{
assert( (x >= 0) && (y >= 0) && ((x > 3) || (y > 3)) );
.....
}
```

Domanda 47

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri un processo software costituito da due fasi F1 ed F2 ciascuna di costo 10000. Con probabilità $p = 0.1$ la fase F1 deve essere ripetuta (a causa di change requests) e con probabilità $(1 - p)$ si passa alla fase F2 e poi al completamento (End) dello sviluppo. Qual'è il costo atteso per lo sviluppo del software seguendo il processo sopra descritto ?

Scegli un'alternativa:

- ☐ 1. 25000
- ☒ 2. 21000 ✓
- ☐ 3. 30000

Your answer is correct.

La risposta corretta è: 21000

Domanda 48

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 5 tasks: T0, T1, T2, T3, T4.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
2	0
2	1
3	0
3	1
4	2
4	3

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	5
1	2
2	1
3	2
4	2

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponendo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.

Time slot (Month)	Task	Effort (PM)
0	0	1
0	1	2
1	0	4
2	2	1
2	3	2
5	4	1
- ☐ 2.

Time slot (Month)	Task	Effort (PM)
1	1	2
2	2	1
4	0	1
5	0	4
6	3	2
8	4	2
- ☒ 3.

Time slot (Month)	Task	Effort (PM)
0	0	1
0	1	2
1	0	4
2	2	1
2	3	2
5	4	2

 Correct.

Your answer is correct.

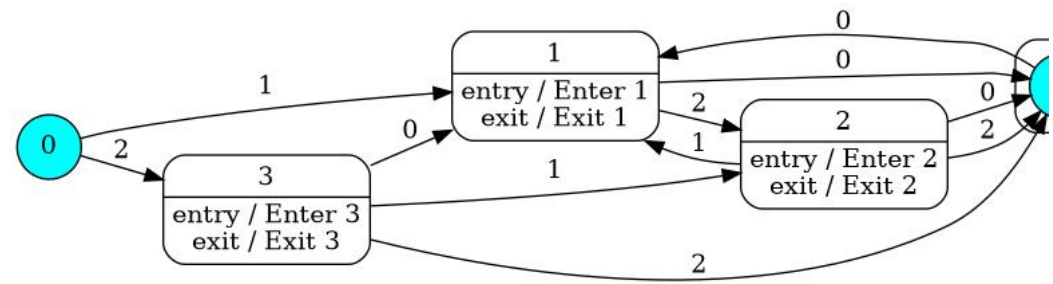
La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	0	1
0	1	2
1	0	4
2	2	1
2	3	2
5	4	2

Domanda 49**Si consideri lo state diagram in figura.**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

**Quale delle seguenti funzioni di transizione meglio lo rappresenta?**

Scegli un'alternativa:

- ☒ 1. `#include "main.h"` ✔ Correct.

```

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (0);
    case 1: return (1);
    case 2: return (3);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (4);
    case 1: return (1);
    case 2: return (2);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (1);
    case 1: return (2);
    case 2: return (4);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (1);
    case 1: return (4);
    case 2: return (4);
    default: return (0); }
  default: return (0);
}
} // f()
  
```

- ☐ 2. `#include "main.h"`
- ```
int f(int x, int u)
{ switch (x) {
 case 0: switch (u) {
 case 0: return (1);
 case 1: return (0);
 case 2: return (4);
 default: return (0); }
 case 1: switch (u) {
 case 0: return (4);
 case 1: return (4);
 case 2: return (1);
 default: return (0); }
 case 2: switch (u) {
 case 0: return (1);
 case 1: return (3);
 case 2: return (3);
 default: return (0); }
 case 3: switch (u) {
 case 0: return (1);
 case 1: return (0);
 case 2: return (0);
 default: return (0); }
 case 4: switch (u) {
 case 0: return (1);
 case 1: return (1);
 case 2: return (4);
 default: return (0); }
 default: return (0);
}
} // f()
```
- ☐ 3. `#include "main.h"`
- ```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (2);
        case 1: return (2);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    default: return (0);
}
} // f()
```

Your answer is correct.

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (2);
        case 2: return (4);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    default: return (0);
} } // f()
```

La risposta corretta è:

Domanda 50

Si consideri la seguente funzione di transizione.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

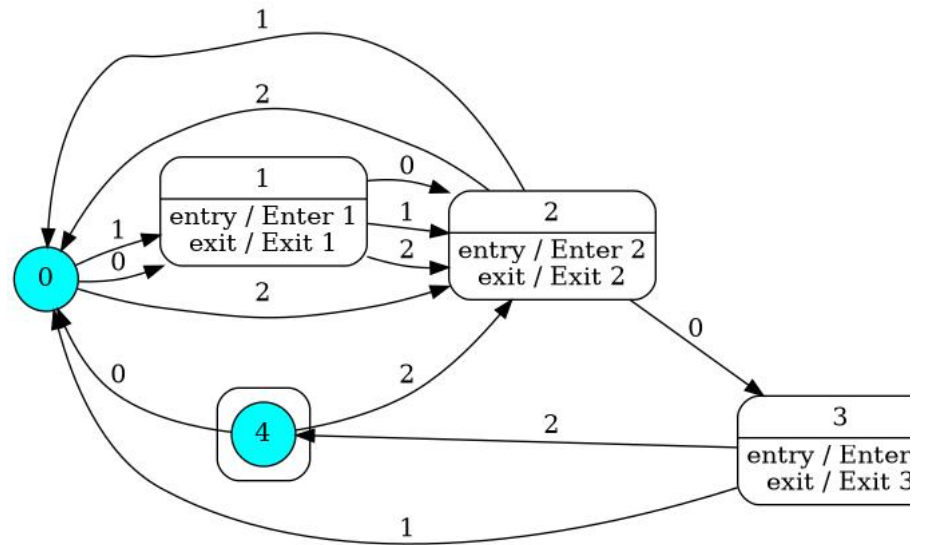
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (0);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (0);
        default: return (0); }
    default: return (0);
} } // f()
```

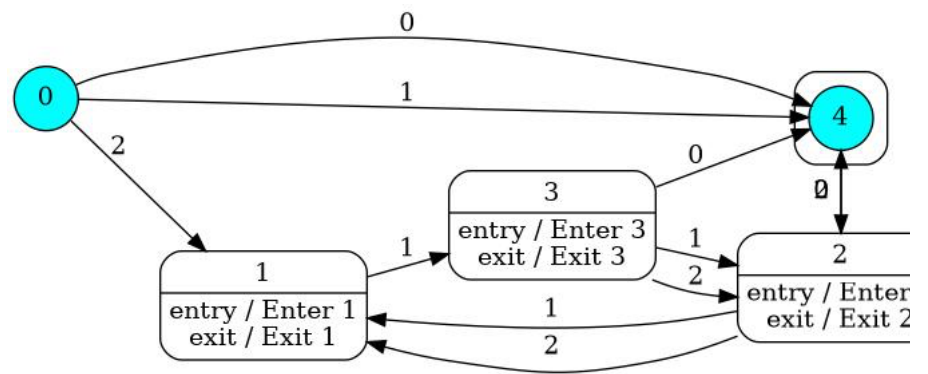
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

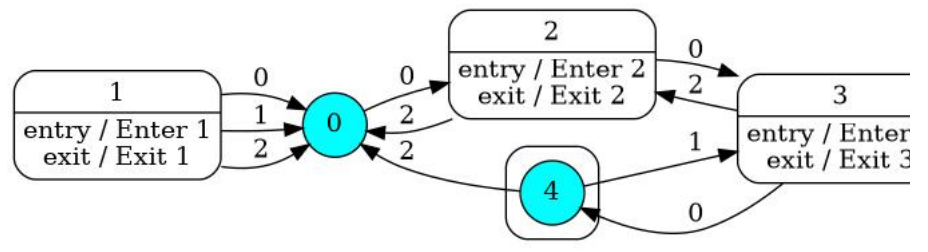
☐ 1.



☐ 2.



☒ 3.



Your answer is correct.

La risposta corretta è:

